

DOCUMENTACION TECNICA EXTENDIDA - SudoAgile

=====

Version del documento: 1.0

Objetivo: documentacion de continuidad para equipos de desarrollo y para alimentar agentes IA de soporte tecnico/funcional.

Alcance: arquitectura, dominio, seguridad, endpoints, flujos, convenciones, operacion local, riesgos y lineamientos de evolucion.

1) VISION TECNICA DEL SISTEMA

SudoAgile es una aplicacion full stack para gestion agil de trabajo, con jerarquia central:

Proyecto -> Modulo -> Tarea -> Subtarea

Capacidades principales:

- autenticacion JWT con control de acceso por roles,
- CRUD + operaciones de flujo para entidades de gestion,
- asociacion de usuarios y prioridades por nivel jerarquico,
- gestion de adjuntos por tipo (documento, producto, marco, etc.),
- archivado y eliminacion logica para proyectos,
- exportacion de proyectos a Excel.

2) ESTRUCTURA DEL REPOSITORIO

Raiz funcional:

C:\SudoProjects\SudoGes\SudoAgile-main

Componentes principales:

- pom.xml (padre Maven multi-modulo)
- application/ (API REST, seguridad, servicios de aplicacion)
- domain/ (entidades, enums y agregados de negocio)
- infraestructure/ (repositorios, DAO, integraciones de persistencia)
- gestor-frontend/ (cliente React + Vite + Ant Design)

Build parent:

- Spring Boot parent 3.1.9
- Java definido en padre: 21
- Modulos declarados: application, domain, infraestructure

3) TECNOLOGIAS Y DEPENDENCIAS CLAVE

Backend:

- Java + Spring Boot
- Spring Security
- Spring Data JPA / Hibernate
- SQL Server JDBC
- JWT (jjwt-api/impl/jackson)
- OpenFeign
- Apache POI (exportacion Excel)

Frontend:

- React 17
- Vite
- Ant Design
- Axios
- react-router-dom
- SweetAlert2
- chart.js / react-chartjs-2

4) ARRANQUE DE LA APLICACION

Main class:

- com.codigo.msregistro.application.ApplicationLauncher

Anotaciones principales:

- @SpringBootApplication
- @EntityScan
- @ComponentScan("com.codigo.msregistro")
- @EnableJpaRepositories
- @EnableFeignClients

Configuracion de perfiles:

- application-local.properties define spring.profiles.active=cloud
- Archivos disponibles:
 - application.properties
 - application-local.properties

- application-cloud.properties

Nota de continuidad:

revisar la estrategia de perfiles para evitar ambigüedad local/cloud.

5) MODELO DE DOMINIO FUNCIONAL

Entidades nucleares:

- Proyecto
- Modulo
- Tarea
- Subtarea
- Usuario
- Prioridad
- AreaFuncional

Entidades de soporte:

- flujos por nivel (proyecto/modulo/tarea/subtarea),
- adjuntos por tipo (aplicativos, marcos, diccionarios, documentos, productos, bases de datos),
- relaciones usuario-rol y asignaciones por entidad.

Estados (enums):

- EstadoProyecto: PENDIENTE, EN_PROGRESO, COMPLETADO, ENTREGADO, ARCHIVADO
- EstadoModulo: PENDIENTE, EN_PROGRESO, COMPLETADO, CANCELADO
- EstadoTarea: PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADO
- EstadoSubtarea: PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADO

Roles (enum Role):

- ADMINISTRADOR
- GESTOR
- DESARROLLADOR
- DICENIADOR
- QA
- DEVOPS
- SOPORTE

6) SEGURIDAD Y AUTENTICACION

Archivo clave:

- application/.../config/SecurityConfiguration.java

Puntos relevantes:

- seguridad stateless (SessionCreationPolicy.STATELESS),
- filtro JWT antes de UsernamePasswordAuthenticationFilter,
- /api/v1/autenticacion/** permitido sin autenticacion,
- autorizacion por hasAnyAuthority en rutas de negocio,
- /api/prioridad/** aparece permitido,
- CORS habilitado via CorsConfigurationSource.

CORS:

- application/.../config/CorsConfig.java
- origenes permitidos por defecto:
 - http://localhost:5173
 - http://127.0.0.1:5173

Autenticacion REST:

- /api/v1/autenticacion/signin
- /api/v1/autenticacion/signupuser
- /api/v1/autenticacion/signupadmin
- /api/v1/autenticacion/validateToken

Convencion frontend:

- token JWT en localStorage ("token")
- axios interceptor agrega Authorization: Bearer <token>

7) FRONTEND - ORGANIZACION Y RUTAS

Archivo principal:

- gestor-frontend/src/App.jsx

Rutas de alto impacto:

- /
- /proyectoList
- /proyectoListArchivados
- /proyectoPapelera
- /usuarioslista
- /areas-funcionales (condicionado a rol ADMINISTRADOR)
- /modulos/:proyectoId/tareas

- /proyectos/:proyectoId/modulos/:moduloId
- /proyectos/:proyectoId/modulos/:moduloId/tarea/:tareaId

Comportamiento de sesion:

- App decodifica payload JWT en cliente,
- valida expiracion,
- fuerza cierre de sesion al vencer token.

8) FRONTEND - CAPA DE SERVICIOS

Ubicacion:

- gestor-frontend/src/services/

Punto base HTTP:

- services/api.js
- baseUrl actual: http://localhost:8080/api
- interceptor para token.

Vite proxy:

- gestor-frontend/vite.config.js
- /api -> http://localhost:8080

Variables usadas en componentes:

- VITE_BACKEND_URL
- VITE_API_URL

Riesgo tecnico actual:

coexisten llamadas via instance(baseUrl /api) y llamadas directas con backendUrl en componentes; conviene unificar estrategia para reducir inconsistencias.

9) CATALOGO DE ENDPOINTS (ALTO NIVEL)

9.1 Autenticacion

- /api/v1/autenticacion/signin
- /api/v1/autenticacion/signupuser
- /api/v1/autenticacion/signupadmin
- /api/v1/autenticacion/validateToken
- /api/v1/autenticacion/claveToken

9.2 Usuarios y roles

- /api/usuarios
- /api/usuarios/getCurrentUser
- /api/usuarios/{id}
- /api/usuarios/{id}/estado
- /api/usuariosrol/todos
- /api/usuariosrol/{id}/roles
- /api/usuariosrol/roles
- /api/rol

9.3 Areas funcionales

- /api/areas-funcionales (GET, POST)

9.4 Prioridades

- /api/prioridad (GET, POST)

9.5 Proyectos

Base:

- /api/proyectos

Operaciones destacadas:

- GET /api/proyectos
- GET /api/proyectos/
- GET /api/proyectos/{id}
- POST /api/proyectos
- POST /api/proyectos/v2
- PATCH /api/proyectos/actualizar/{id}
- PATCH /api/proyectos/actualizarFechaInicio/{idProyecto}
- PATCH /api/proyectos/{idProyecto}/ampliar/dias={dias}
- PUT /api/proyectos/{proyectoId}/actualizar-estado
- PUT /api/proyectos/actualizar-posicion
- PUT /api/proyectos/{id}/estado
- PUT /api/proyectos/{id}/restaurar
- PATCH /api/proyectos/{id}/eliminar
- PATCH /api/proyectos/{id}/restaurar-papelera
- GET /api/proyectos/papelera
- GET /api/proyectos/eliminados
- GET /api/proyectos/no-archivados
- GET /api/proyectos/no-archivados-por-gestor/{id}
- GET /api/proyectos/archivados
- GET /api/proyectos/descargar

- PUT /api/proyectos/{id}/usuarios
- DELETE /api/proyectos/{proyectoId}/usuarios/{usuarioId}
- PUT /api/proyectos/{idProyecto}/prioridad/{idPrioridad}
- PUT /api/proyectos/{idProyecto}/prioridad

9.6 Modulos

Base:

- /api/proyectosmodulo/{proyectoId}/modulos

Operaciones destacadas:

- POST /api/proyectosmodulo/{proyectoId}/modulos
- GET /api/proyectosmodulo/{proyectoId}/modulos/{moduloId}
- PATCH /api/proyectosmodulo/{proyectoId}/modulos/actualizar/{moduloId}
- PATCH /api/proyectosmodulo/{proyectoId}/modulos/actualizarFechaInicio/{moduloId}
- PATCH /api/proyectosmodulo/{proyectoId}/modulos/actualizarFechaFin/{moduloId}
- PUT /api/proyectosmodulo/{proyectoId}/modulos/actualizar-posicion
- PUT /api/proyectosmodulo/{proyectoId}/modulos/{moduloId}/estado
- DELETE /api/proyectosmodulo/{proyectoId}/modulos/delete/{idModulo}
- PUT /api/proyectosmodulo/{proyectoId}/modulos/{id}/usuarios
- DELETE /api/proyectosmodulo/{proyectoId}/modulos/{moduloId}/usuarios/{usuarioId}
- PUT /api/proyectosmodulo/{proyectoId}/modulos/{idModulo}/prioridad/{idPrioridad}
- PUT /api/proyectosmodulo/{proyectoId}/modulos/{idModulo}/prioridad

9.7 Tareas

Base:

- /api/modulos/{moduloId}/tareas

Operaciones destacadas:

- POST /api/modulos/{moduloId}/tareas
- GET /api/modulos/{moduloId}/tareas
- GET /api/modulos/{moduloId}/tareas/{tareaId}
- PATCH /api/modulos/{moduloId}/tareas/actualizar/{tareaId}
- PUT /api/modulos/{moduloId}/tareas/actualizar/{tareaId}
- PATCH /api/modulos/{moduloId}/tareas/actualizarFechaInicio/{tareaId}
- PATCH /api/modulos/{moduloId}/tareas/actualizarFechaFin/{tareaId}
- PUT /api/modulos/{moduloId}/tareas/actualizar-posicion
- PUT /api/modulos/{moduloId}/tareas/{tareaId}/estado
- DELETE /api/modulos/{moduloId}/tareas/delete/{idTarea}
- PUT /api/modulos/{moduloId}/tareas/{id}/usuarios
- DELETE /api/modulos/{moduloId}/tareas/{tareaId}/usuarios/{usuarioId}
- PUT /api/modulos/{moduloId}/tareas/{idTarea}/prioridad/{idPrioridad}
- PUT /api/modulos/{moduloId}/tareas/{tareaId}/prioridad

9.8 Subtareas

Base:

- /api/tareas/{tareaId}/subTareas

Operaciones destacadas:

- POST /api/tareas/{tareaId}/subTareas
- PATCH /api/tareas/{tareaId}/subTareas/actualizar/{subtareaId}
- PATCH /api/tareas/{tareaId}/subTareas/actualizarFechaInicio/{subtareaId}
- PATCH /api/tareas/{tareaId}/subTareas/actualizarFechaFin/{subtareaId}
- PUT /api/tareas/{tareaId}/subTareas/actualizar-posicion
- PUT /api/tareas/{tareaId}/subTareas/{subtareaId}/estado
- DELETE /api/tareas/{tareaId}/subTareas/delete/{subtareaId}
- PUT /api/tareas/{tareaId}/subTareas/{subtareaId}/usuarios
- DELETE /api/tareas/{tareaId}/subTareas/{subtareaId}/usuarios/{usuarioId}
- PUT /api/tareas/{tareaId}/subTareas/{idSubtarea}/prioridad/{idPrioridad}
- PUT /api/tareas/{tareaId}/subTareas/{idSubtarea}/prioridad

9.9 Flujos

- /api/flujos/proyectos/*
- /api/flujos/modulos/*
- /api/flujos/tareas/*
- /api/flujos/subtareas/*

Patrones:

- mostrar/{entidadId}/{usuarioId}
- update/descripcion/{flujoId}
- update/progreso/{flujoId}
- update/fechaA/{flujoId}

9.10 Adjuntos y artefactos por tipo

Bases de ruta:

- /api/aplicativos/{proyectos|modulos|tareas|subtareas}
- /api/marcos/{proyectos|modulos|tareas|subtareas}
- /api/diccionarios/{proyectos|modulos}
- /api/documentos/{proyectos|modulos}
- /api/productos/{proyectos|modulos}
- /api/basesdato/{proyectos|modulos}

Patrones recurrentes:

- GET /todo/{id}
- GET /{id}/descargar
- POST multipart/form-data

10) FLUJOS DE NEGOCIO RELEVANTES

10.1 Ciclo principal

1. crear proyecto,
2. crear modulos,
3. crear tareas,
4. crear subtareas,
5. asignar usuarios y prioridad,
6. mover estados y actualizar fechas,
7. seguimiento via flujos.

10.2 Cierre de proyecto

- opcion A: estado ENTREGADO/COMPLETADO
- opcion B: ARCHIVADO
- opcion C: eliminacion logica (papelera) y eventual restauracion.

10.3 Gobierno de permisos

- endpoints protegidos por role/authority en SecurityConfiguration.
- UI tambien filtra por rol para mostrar/ocultar rutas.

11) EJECUCION EN LOCAL

Prerequisitos:

- Java 21
- Maven 3.9+
- Node.js LTS + npm
- SQL Server accesible

Backend:

1. cd C:\SudoProjects\SudoGes\SudoAgile-main
2. mvn clean install
3. mvn -pl application spring-boot:run

Frontend:

1. cd C:\SudoProjects\SudoGes\SudoAgile-main\gestor-frontend
2. npm install
3. npm run dev

Puertos habituales:

- API: 8080
- Frontend: 5173

12) CONFIGURACION Y AMBIENTES

Datasource y seguridad se encuentran en properties.

Lineamientos:

- externalizar secretos por ambiente,
- no hardcodear tokens/credenciales,
- parametrizar URLs backend/frontend por entorno.

Checklist ambiente:

- [] URL datasource correcta
- [] usuario/password BD por ambiente
- [] key.signature por ambiente
- [] CORS origenes correctos
- [] URL frontend/backend coherentes

13) CONVENCIONES DE CODIGO Y EVOLUCION

Backend:

- mantener flujo controller -> service -> repository,
- separar DTOs de entidades cuando aplique,
- evitar logica de negocio en controlador,
- estandarizar respuestas de error.

Frontend:

- concentrar llamadas HTTP en services/,
- minimizar llamadas directas en componentes,
- usar hooks para logica reusable,
- mantener naming consistente (es/es-EN) para reducir deuda.

Cross-cutting:

- evitar duplicacion de endpoints o variantes ambiguas,
- centralizar manejo de errores de red/auth.

14) OBSERVACIONES TECNICAS PARA CONTINUIDAD

Hallazgos relevantes:

- coexistencia de estilos de consumo HTTP (instance y axios directo),
- rutas con nomenclaturas heterogeneas (camelCase/mixtas),
- algunos controladores mezclan mensajes y estructuras de respuesta distintas.

Recomendaciones de mejora incremental:

1. estandarizar contrato de error (code, message, details),
2. unificar cliente HTTP frontend,
3. documentar OpenAPI para estabilizar integraciones,
4. alinear estrategia de perfiles local/cloud,
5. normalizar convenciones de nombres en rutas y payloads.

15) GUIA PARA AGENTE IA TECNICO (IN-APP O SOPORTE)

15.1 Intenciones tecnicas que debe identificar

- "no puedo iniciar sesion"
- "me devuelve 401/403"
- "no se ven proyectos para gestor"
- "como archivar/restaurar proyecto"
- "como asignar usuarios a modulo/tarea/subtarea"
- "como cargar o descargar adjuntos"
- "como actualizar prioridad/estado/fecha"
- "que endpoint corresponde a X"
- "por que no aparece opcion en menu"

15.2 Respuesta esperada del agente tecnico

- explicar primero causa probable,
- luego accion inmediata,
- luego verificacion final,
- incluir ruta funcional o endpoint cuando sea util,
- mencionar restriccion por rol cuando corresponda.

15.3 Reglas de seguridad para el agente

- no exponer secretos ni credenciales,
- no sugerir bypass de autorizacion,
- no inventar endpoints,
- usar solo rutas documentadas.

15.4 Plantillas utiles para respuestas IA

- "Valida si tu token sigue vigente. Si expiro, inicia sesion de nuevo y reintenta la operacion."
- "Para asignar usuarios a un proyecto usa la operacion de usuarios del recurso proyecto y envia lista de IDs."
- "Si tu rol no incluye permisos administrativos, algunas rutas/menu estaran ocultos por diseno."

16) RUNBOOK DE INCIDENTES

Incidente: 401/403 recurrente

Pasos:

1. validar token en frontend,
2. revisar expiracion del JWT,
3. confirmar autoridad del rol en SecurityConfiguration,
4. revisar que endpoint sea el esperado.

Incidente: CORS en desarrollo

Pasos:

1. validar origen frontend (5173),
2. revisar CorsConfig,
3. confirmar uso de URL/proxy correcto.

Incidente: datos no visibles por perfil

Pasos:

1. validar roles del usuario con /api/usuarios/getCurrentUser,
2. validar filtros por gestor/area funcional,
3. revisar endpoint usado en frontend.

Incidente: falla en adjuntos

Pasos:

1. confirmar endpoint de tipo correcto (documento/producto/etc.),
2. validar envio multipart/form-data,
3. revisar permisos y payload.

17) PRUEBAS Y VALIDACION RECOMENDADAS

Antes de merge:

- [] login/logout con roles distintos
- [] CRUD proyecto/modulo/tarea/subtarea
- [] asignacion/quita de usuarios
- [] cambio de prioridad y estado

- [] archivado/restauracion/papelera
- [] flujos (descripcion/progreso/fecha)
- [] carga y descarga de adjuntos
- [] build frontend y backend sin errores

18) ROADMAP TECNICO SUGERIDO

Corto plazo:

1. unificacion de capa HTTP frontend,
2. normalizacion de errores API,
3. eliminar secretos de configuraciones versionadas.

Mediano plazo:

1. especificacion OpenAPI,
2. suite de pruebas automatizadas por capas,
3. observabilidad estandar (logs estructurados y trazas).

Largo plazo:

1. modularizacion de frontend por dominio,
2. endurecimiento de seguridad y gestion de secretos por vault,
3. analitica operativa y reporting de productividad.

19) MATRIZ RAPIDA DE ARCHIVOS CLAVE

Backend:

- application/src/main/java/.../ApplicationLauncher.java
- application/src/main/java/.../config/SecurityConfiguration.java
- application/src/main/java/.../config/CorsConfig.java
- application/src/main/java/.../controller/*.java
- application/src/main/resources/application*.properties

Domain:

- domain/src/main/java/.../aggregates/*.java
- enums de estado y roles en aggregates

Frontend:

- gestor-frontend/src/App.jsx
- gestor-frontend/src/components/*
- gestor-frontend/src/services/api.js
- gestor-frontend/src/services/*
- gestor-frontend/vite.config.js

20) RESUMEN DE CONTINUIDAD

Este sistema ya tiene una base funcional amplia de gestion agil.

La prioridad para siguientes equipos debe ser:

- estabilizar contratos API,
- consolidar estrategia de configuracion por ambiente,
- estandarizar capa frontend de consumo de servicios,
- reforzar trazabilidad y calidad automatizada.

Con este documento, un agente de IA puede asistir en:

- soporte funcional al usuario,
- soporte tecnico de primer nivel,
- orientacion de mantenimiento evolutivo.